

Hash & Encoding Toolkit

A small, fast, single-file web app for hashing, message authentication and encoding. No install, no server, no dependencies. Open the HTML file and it works.

Built by King Sabatsu Kisuke as part of a growing cybersecurity toolkit.

Features

Tab	What it does
Hash	MD5, SHA-1, SHA-256, SHA-384, SHA-512 of any text, live as you type. One-tap copy and a verify box that tells you which algorithm a pasted hash matches.
HMAC	Keyed hashes (HMAC-SHA-1/256/384/512) from a message and secret key. Paste an expected signature to check it.
File	Drop or choose a file and get all five hashes. Verify a download against a published checksum. Limit: 200 MB.

Tab	What it does
Encode	Base64, Base64 URL-safe, Base32, URL, Hex, Binary, Octal, HTML entities, Unicode escapes, Morse, ROT13, and Caesar cipher with a custom shift (-25 to 25) plus a "try all 25 shifts" brute-force decoder.

Usage

1. Open `hash-encoding-toolkit.html` in any modern browser, or use the published link.
2. Pick a tab and enter your input. Results update instantly.
3. In **Encode**, use Encode or Decode, and Swap to feed the output back in as input.

Example walkthroughs

Verify a download. Open File, drop the file, then paste the checksum from the publisher's website into the verify box. A green "Match" means the file is intact.

Check a webhook signature. Open HMAC, paste the request body as the message and your shared secret as the key, choose the algorithm (often SHA-256), then paste the signature from the request header. A match means it is authentic.

Crack a Caesar cipher. Open Encode, choose Caesar, paste the ciphertext and press "Try all 25 shifts". Read down the list for the line that makes sense.

Known test values

Input	Result
MD5 of hello	5d41402abc4b2a76b9719d911017c592
SHA-256 of hello	starts with 2cf24dba
Base64 of hello	aGVsbG8=
Caesar shift 3 of Hello, World!	Khoor , Zruog!
HMAC- SHA- 256, key key , message The quick	f7bc83f4...3cd8

Input	Result
brown fox jumps over the lazy dog	

How it works

- **SHA and HMAC** use the browser's built-in Web Crypto API (`crypto.subtle`).
- **MD5** is not offered by Web Crypto, so a short implementation is included in the page.
- Text is converted to UTF-8 bytes first, so accents and emoji hash and encode correctly.
- **Files** are read into memory with `File.arrayBuffer()` and hashed in one pass, which is why there is a size limit.
- Decoders validate input and show an error instead of garbage when the data does not fit the format.

Privacy

Everything runs locally in your browser. Files and text are never uploaded, stored or logged.

Security notes

- Hashing is one-way. Encoding is reversible.
Encoding is not encryption. Base64, Hex, ROT13, Caesar and the rest protect nothing.
- MD5 and SHA-1 are cryptographically broken. Use them for checksums and learning, never for signatures or passwords.
- HMAC needs a secret key to be meaningful. Keep keys out of screenshots and shared links.
- To store passwords, use Argon2, bcrypt or scrypt, not a plain hash.
- A matching hash proves integrity only if the checksum itself came from a trusted source.

Limitations

- Files over 200 MB are refused to avoid running out of browser memory.
- HMAC-MD5 is not available (Web Crypto does not support it).
- Caesar and ROT13 only shift the 26 English letters. Other characters pass through unchanged.
- Morse supports A-Z and 0-9 only.

Project structure

File	Purpose
hash-encoding-toolkit.html	The entire app (HTML, CSS and JS)
README.md	This file

Roadmap

- Base58 and Vigenere cipher
- JWT decoder
- Streaming hashes for very large files
- Password strength and breach-style checks
- Port to the Python desktop suite

Contributing and feedback

Found a bug or have an idea? Open an issue or send a note. This project is a learning build, so feedback that makes it better is welcome.